

Compression and Decompression Techniques

04 July 2026 20:23

Compression is the process of reducing the size of data by removing redundancy so that it occupies less storage space and can be transmitted faster.

Decompression is the reverse process where the compressed data is stored to its original form.

Compression techniques are widely used in:

- File archiving(.zip, .rar, .7z)
- Images(.jpg, .png)
- Audio (.mp3)
- Video (.mp4)
- Data transmission over networks

Types of Compression

There are two major categories of compression:

Compression Type	Description	Example
Lossless Compression	Original data can be reconstructed exactly.	Zip, png, Huffman Coding, Run-length Encoding
Lossy Compression	Some data is permanently lost for higher compression.	jpeg, mp3, mpeg

Since C is commonly used for system programming, most compression algorithms implemented in C are lossless.

Basic Compression Techniques

1. Run-Length Encoding(RLE)

Run - length encodings replaces consecutive repeated characters with a single character followed by its count.

Example:

Original Data:

AAAAAABBBCCCCCCCCDDDDDD

Compressed data:

A6B3C7D6

home work:-
WAP in C to compress and decompress
using R-L Encoding (RLE)

Compression Algorithm

- i. Read the first character.
- ii. Count consecutive occurrences
- iii. Write the character and its count.
- iv. Continue until end of the file.

RLE Decompression Algorithm

- i. Read one character
- ii. Read its count
- iii. Print the character count times
- iv. Continue until EOF

Advantage of RLE

- Very easy to implement
- Fast
- Excellent for repetitive data

Disadvantage:

Suppose the file contains:

ABCDEFGF

Compressed version:

A1B1C1D1E1F1G1

This is larger than the original file.

Huffman Coding

Huffman coding is a variable length coding technique.

Frequently occurring character receives shorter binary codes.

Rare character receives longer binary codes.

Example

Character	Frequency	Code
A	50	0
B	30	10
C	15	110
D	5	111

D | E | C | B | A

5 | 10 | 15 | 30 | 45

Original:

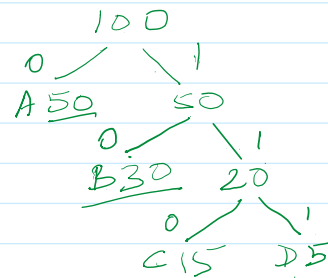
ABCD

Binary:

010110111

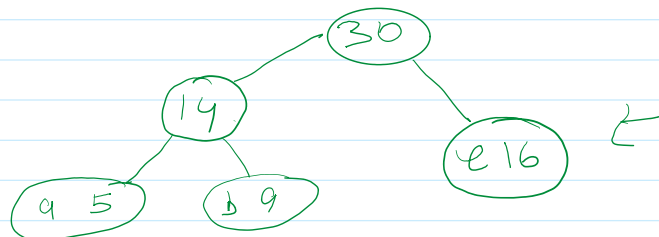
Steps:

1. Count frequency of each character
2. Create leaf nodes
3. Build Huffman Tree
4. Generate binary codes
5. Encode the file.
6. Decode using the same tree.

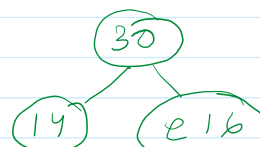
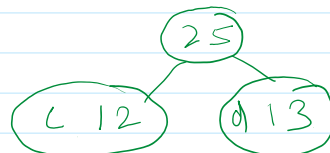


Code becomes
 A = 0 1
 B = 10 2
 C = 110 3
 D = 111 3

Characters	Frequencies
a	5
b	9
c	12
d	13
e	16
f	45



c 12
 d 13
 Internal → 14
 e 16
 f 45



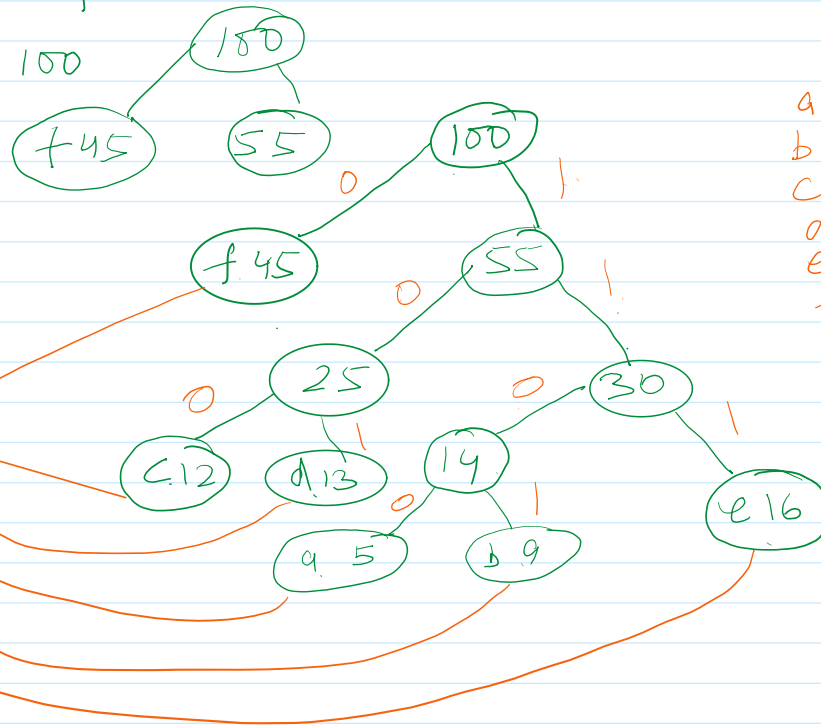
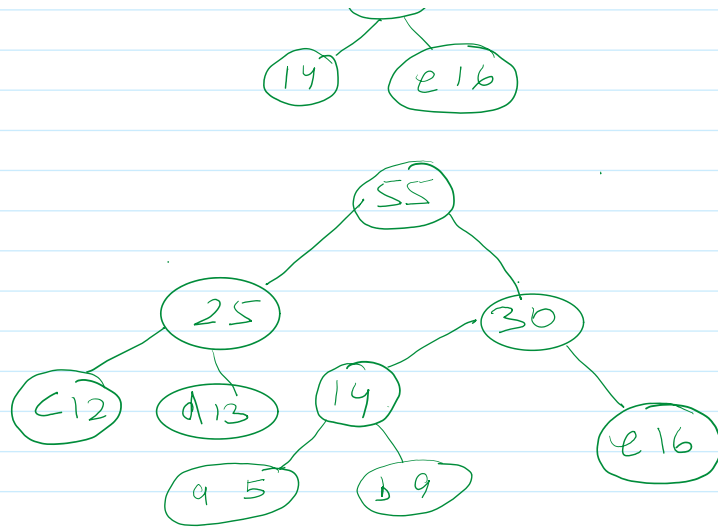
f 100
45

internal → 14 }
e → 16 }
internal → 25 }
f → 45 }

internal → 25 }
internal → 30 }
f → 45 }

internal → 55 }
f → 45 }

internal → 100



a → 1100
b → 1101
c → 100
d → 101
e → 111
f → 0

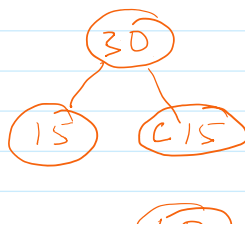
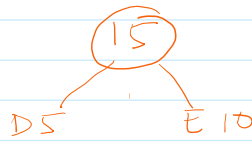
leaf nodes
they are
containing
data.

D	E	C	B	A
---	---	---	---	---

5	10	15	30	45
---	----	----	----	----

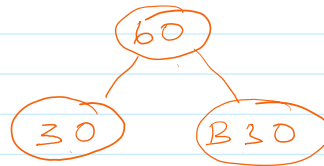
D 5 }
E 10 }
C 15 }
B 30 }
A 45 }

internal 15 }
C 15 }
R 20 }



Internal 15 }
 C 15 }
 B 30
 A 45

(15) (15)



Internal 30 }
 B 30 }
 A 45

A 45 }
 Internal 60 }

A = 0
 B = 11
 C = 101
 D = 1000
 E = 1001

